



Pemrograman Java — Pertemuan 2

Selamat datang di sesi kedua pembelajaran Java! Pada pertemuan ini kita akan membahas dua topik utama yang menjadi fondasi penting dalam membangun program Java yang interaktif dan fungsional.

Pengantar Aplikasi Java

Struktur program, output, dan kompilasi

Input/Output & Operator

Membaca input, aritmatika, dan pengambilan keputusan



Pengenalan Pemrograman Java

Kita memulai perjalanan belajar Java dengan memahami struktur dasar sebuah program. Pada pertemuan ini, kita akan melihat bagaimana Java menampilkan pesan ke layar, membaca input dari pengguna, dan mengambil keputusan berdasarkan kondisi tertentu.

Output ke Layar

Program pertama menampilkan pesan sederhana menggunakan `System.out.println` — fondasi komunikasi program dengan pengguna.

Membaca Input & Menghitung

Program kedua meminta dua angka dari pengguna, menghitung jumlahnya, lalu menampilkan hasilnya secara terformat.

Membandingkan Angka

Contoh ketiga membandingkan dua nilai menggunakan operator relasional dan menampilkan pesan hasil perbandingan.



Contoh Pertama: Mencetak Baris Teks

Program Java paling sederhana adalah program yang mencetak satu baris teks ke layar. Ini adalah titik awal yang sempurna untuk memahami struktur dasar sebuah aplikasi Java.

```
// Fig. 2.1: Welcome1.java
// Text-printing program.

public class Welcome1 {
    // main method begins execution of Java application
    public static void main(String[] args) {
        System.out.println("Welcome to Java Programming!");
    } // end method main
} // end class Welcome1
```

public class Welcome1

Setiap program Java harus berada di dalam sebuah **class**. Nama class harus sama dengan nama file (`Welcome1.java`).

public static void main(String[] args)

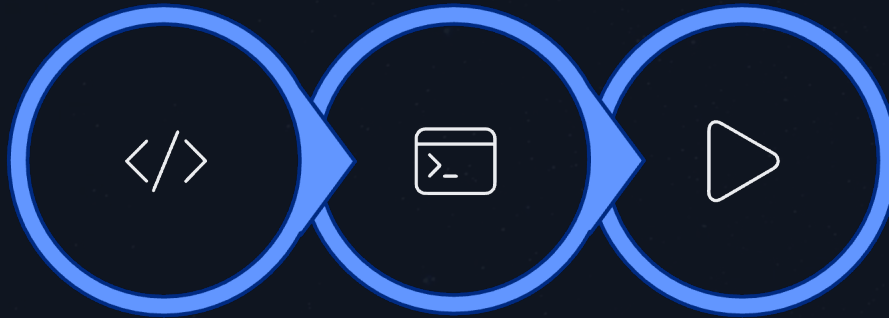
Titik awal eksekusi program. JVM selalu memanggil metode `main` pertama kali saat program dijalankan.

System.out.println(...)

Perintah untuk mencetak teks ke layar, diikuti dengan baris baru secara otomatis setelah output ditampilkan.

Melakukan Kompilasi Program Java

Sebelum program Java dapat dijalankan, kode sumber (`.java`) harus dikompilasi menjadi *bytecode* (`.class`) menggunakan Java Development Kit (JDK). Proses ini menggunakan alat baris perintah, bukan IDE.



Tulis kode

Kompilasi

Jalankan

Perintah Kompilasi

```
javac Welcome1.java
```

Menghasilkan file `Welcome1.class` berisi bytecode yang siap dieksekusi oleh JVM.

Perintah Menjalankan Program

```
java Welcome1
```

JVM (Java Virtual Machine) membaca bytecode dan menjalankan program. Output akan muncul di terminal.



Memodifikasi Program Java Pertama

Setelah berhasil menjalankan program pertama, langkah selanjutnya adalah mencoba memodifikasinya. Ini adalah cara terbaik untuk memahami bagaimana setiap bagian kode bekerja dan apa pengaruhnya terhadap output program.

LAPTOP



Versi 1: Satu Baris Output

Menggunakan `System.out.println` untuk menampilkan satu kalimat. Setelah teks dicetak, kursor berpindah ke baris berikutnya secara otomatis.



Versi 2: Beberapa Baris Output

Menggunakan karakter escape `\n` untuk membuat baris baru di dalam satu perintah `System.out.print`, atau menggunakan beberapa perintah `println` secara berurutan.

- ① Karakter escape seperti `\n` (baris baru), `\t` (tab), dan `\\` (backslash) memungkinkan kita mengontrol format output tanpa menulis beberapa perintah print.

Menampilkan Teks dengan printf

Metode `System.out.printf` (huruf `f` singkatan dari *"formatted"*) memberikan kontrol yang lebih presisi dalam menampilkan data. Berbeda dengan `println`, `printf` menggunakan **format string** dan **format specifier** untuk mengatur tampilan output.

%d

Format specifier untuk bilangan bulat (*integer*). Contoh: `printf("%d", 42) → 42`

%f

Format specifier untuk bilangan desimal (*float/double*). Contoh: `printf("%.2f", 3.14159) → 3.14`

%s

Format specifier untuk string (teks). Contoh: `printf("%s", "Hello") → Hello`

%n

Karakter baris baru yang portabel, lebih disarankan dibanding `\n` saat menggunakan `printf`.

```
System.out.printf("Sum is %d%n", sum); // menampilkan nilai sum
```

Aplikasi Sederhana: Menambahkan Integer



Program berikut membaca **dua bilangan bulat** yang diketik pengguna di keyboard, menghitung jumlahnya, lalu menampilkan hasilnya. Ini memperkenalkan konsep penting: bagaimana program *mengingat* data menggunakan variabel.

1 Import Scanner

Kelas `java.util.Scanner` memungkinkan program membaca input dari berbagai sumber, termasuk keyboard (`System.in`).

2 Deklarasi Variabel

Variabel seperti `number1`, `number2`, dan `sum` menyimpan nilai di memori komputer. Setiap variabel memiliki **nama**, **tipe data**, **ukuran**, dan **nilai**.

3 Proses & Output

Program menjumlahkan kedua angka (`sum = number1 + number2`) lalu menampilkan hasilnya menggunakan `printf`.

Kode Program: Adding Integers

Berikut adalah kode lengkap program penjumlahan dua bilangan bulat. Perhatikan bagaimana setiap bagian program bekerja bersama untuk membaca input, memproses, dan menampilkan output.

```
// Fig. 2.7: Addition.java
// Addition program that inputs two numbers then displays their sum.
import java.util.Scanner; // program uses class Scanner

public class Addition {
    // main method begins execution of Java application
    public static void main(String[] args) {
        // create a Scanner to obtain input from the command window
        Scanner input = new Scanner(System.in);

        System.out.print("Enter first integer: "); // prompt
        int number1 = input.nextInt(); // read first number from user

        System.out.print("Enter second integer: "); // prompt
        int number2 = input.nextInt(); // read second number from user

        int sum = number1 + number2; // add numbers, store total in sum

        System.out.printf("Sum is %d\n", sum); // display sum
    } // end method main
} // end class Addition
```

📌 Gunakan `input.nextInt()` untuk membaca bilangan bulat. Untuk bilangan desimal, gunakan `input.nextDouble()`. Jangan lupa selalu memberi **prompt** sebelum membaca input agar pengguna tahu apa yang harus diketik!

Konsep Memori (Memory Concepts)

Ketika kita mendeklarasikan variabel dalam Java, komputer mengalokasikan sebuah lokasi di memori untuk menyimpan nilainya. Nama variabel seperti `number1`, `number2`, dan `sum` adalah label yang merujuk ke lokasi memori tersebut.

Properti Sebuah Variabel

Nama

Identifier yang digunakan untuk mengakses nilai, misal: `number1`

Tipe Data

Menentukan jenis data yang bisa disimpan, misal: `int`, `double`, `String`

Ukuran

Ruang memori yang dibutuhkan, misal: `int` = 4 bytes

Nilai

Data aktual yang tersimpan, dapat berubah selama program berjalan

```
1 // Fig. 2.7: Addition.java
2 // Addition program that inputs two numbers then displays their sum.
3 import java.util.Scanner; // program uses class Scanner
4
5 public class Addition {
6     // main method begins execution of Java application
7     public static void main(String[] args) {
8         // create a Scanner to obtain input from the command window
9         Scanner input = new Scanner(System.in);
10
11         System.out.print("Enter first integer: "); // prompt
12         int number1 = input.nextInt(); // read first number from user
13
14         System.out.print("Enter second integer: "); // prompt
15         int number2 = input.nextInt(); // read second number from user
16
17         int sum = number1 + number2; // add numbers, then store total in sum
18
19         System.out.printf("Sum is %d\n", sum); // display sum
20     } // end method main
21 } // end class Addition
```

Fig. 2.7 | Addition program that inputs two numbers then displays their sum. (Part 1 of 2.)

Aritmatika dalam Java

Sebagian besar program melakukan perhitungan aritmatika. Java menyediakan operator aritmatika standar untuk operasi matematika dasar. Penting untuk memahami operator mana yang digunakan dan bagaimana Java menangani urutan prioritas operasi (*operator precedence*).

Operasi	Operator Java	Contoh Ekspresi	Hasil
Penjumlahan	<code>+</code>	<code>f + 7</code>	f ditambah 7
Pengurangan	<code>-</code>	<code>p - c</code>	p dikurangi c
Perkalian	<code>*</code>	<code>b * m</code>	b dikali m
Pembagian	<code>/</code>	<code>x / y</code>	x dibagi y
Sisa Bagi	<code>%</code>	<code>r % s</code>	Sisa dari r/s

① Urutan prioritas operator: perkalian, pembagian, dan sisa bagi dieksekusi **sebelum** penjumlahan dan pengurangan. Gunakan tanda kurung `()` untuk mengubah urutan eksekusi sesuai kebutuhan.

Pengambilan Keputusan: Operator Kesetaraan & Relasional

Kondisi adalah ekspresi yang bisa bernilai **benar (true)** atau **salah (false)**. Pernyataan `if` di Java memungkinkan program membuat keputusan berdasarkan nilai kondisi tersebut.

Contoh kondisi: *"Nilai lebih besar dari atau sama dengan 60"* → menentukan apakah seorang siswa lulus ujian. Jika kondisi bernilai `true`, tubuh pernyataan `if` dieksekusi. Jika `false`, tubuhnya dilewati.

Operator Kesetaraan

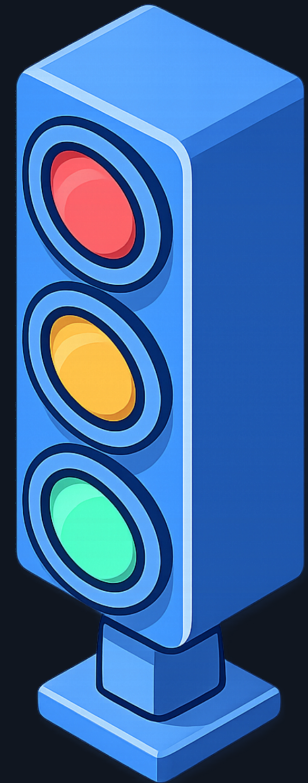
`==` → sama dengan
`!=` → tidak sama dengan

Operator Relasional

`<` → kurang dari
`>` → lebih dari
`<=` → kurang dari atau sama
`>=` → lebih dari atau sama

Sintaks if Statement

```
if (kondisi) {  
    // kode dieksekusi  
    // jika kondisi true  
}
```



Tabel Operator Kesetaraan & Relasional

Berikut adalah ringkasan lengkap operator kesetaraan dan relasional yang tersedia di Java, beserta contoh penggunaannya dalam kondisi `if`. Operator-operator ini adalah alat utama untuk membandingkan nilai dalam pengambilan keputusan.

Operator Java	Operator Aljabar	Contoh Kondisi Java	Arti
<code>==</code>	<code>=</code>	<code>x == y</code>	x sama dengan y
<code>!=</code>	<code>≠</code>	<code>x != y</code>	x tidak sama dengan y
<code>></code>	<code>></code>	<code>x > y</code>	x lebih besar dari y
<code><</code>	<code><</code>	<code>x < y</code>	x lebih kecil dari y
<code>>=</code>	<code>≥</code>	<code>x >= y</code>	x lebih besar atau sama dengan y
<code><=</code>	<code>≤</code>	<code>x <= y</code>	x lebih kecil atau sama dengan y

⚠ Jangan keliru antara `==` (operator perbandingan) dan `=` (operator penugasan/assignment)! Menggunakan `=` di dalam kondisi `if` adalah kesalahan umum yang sering terjadi pada pemula.

Contoh Program: IF Statement & Operator Relasional

Program `Comparison.java` membandingkan dua bilangan bulat menggunakan pernyataan `if` dan semua operator relasional. Program membaca dua angka dari pengguna, lalu menampilkan hasil perbandingan yang benar.

```
1 // Fig. 2.15: Comparison.java
2 // Compare integers using if statements, relational operators
3 // and equality operators.
4 import java.util.Scanner; // program uses class Scanner
5
6 public class Comparison {
7     // main method begins execution of Java application
8     public static void main(String[] args) {
9         // create Scanner to obtain input from command line
10        Scanner input = new Scanner(System.in);
```

→ Import & Deklarasi Class

Mengimpor `java.util.Scanner` dan mendeklarasikan class `Comparison` sebagai wadah program.

→ Membuat Scanner & Membaca Input

Membuat objek `Scanner` dan membaca dua bilangan bulat dari pengguna menggunakan `nextInt()`.

→ Sederetan Pernyataan if

Masing-masing `if` memeriksa satu kondisi relasional dan mencetak pesan yang sesuai jika kondisi tersebut benar.

Kode Lengkap: Comparison.java (Bagian 2)

Bagian kedua kode program `Comparison.java` menampilkan semua kemungkinan perbandingan antara dua bilangan bulat menggunakan enam pernyataan `if` terpisah, masing-masing memeriksa satu relasi.

```
11
12     System.out.print("Enter first integer: "); // prompt
13     int number1 = input.nextInt(); // read first number from user
14
15     System.out.print("Enter second integer: "); // prompt
16     int number2 = input.nextInt(); // read second number from user
17
18     if (number1 == number2)
19         System.out.printf("%d == %d\n", number1, number2);
20 }
21
22     if (number1 != number2) {
23         System.out.printf("%d != %d\n", number1, number2);
24     }
25
26     if (number1 < number2) {
27         System.out.printf("%d < %d\n", number1, number2);
28     }
29
30     if (number1 > number2) {
31         System.out.printf("%d > %d\n", number1, number2);
32     }
33
34     if (number1 <= number2) {
35         System.out.printf("%d <= %d\n", number1, number2);
36     }
37
38     if (number1 >= number2) {
39         System.out.printf("%d >= %d\n", number1, number2);
40     }
41 } // end method main
42 } // end class Comparison
```

- ❏ Program menggunakan format string `"%d == %d\n"` di dalam `printf` untuk menampilkan kedua nilai variabel sekaligus bersama operator yang berlaku, sehingga output lebih informatif dan mudah dipahami.

Latihan: Kembangkan Program dari Rumus Berikut

Sekarang saatnya mempraktikkan apa yang telah dipelajari! Buatlah program Java yang menghitung nilai dari rumus-rumus matematika berikut. Gunakan variabel, operator aritmatika, dan tampilkan hasilnya menggunakan `printf`.

1

Luas Lingkaran

Hitung luas lingkaran dengan jari-jari yang diinput pengguna.

Rumus: $L = \pi \times r^2$

Gunakan `Math.PI` untuk nilai π dan `Math.pow(r, 2)` untuk perpangkatan.

2

Konversi Suhu

Konversi suhu dari Celsius ke Fahrenheit berdasarkan input pengguna.

Rumus: $F = (C \times \frac{9}{5}) + 32$

Perhatikan penggunaan tipe data `double` untuk hasil yang akurat!

3

Nilai Akhir Mahasiswa

Hitung nilai akhir berdasarkan tiga komponen penilaian yang diinput pengguna.

Rumus: $NA = (0.3 \times T) + (0.3 \times UTS) + (0.4 \times UAS)$

Tentukan juga apakah mahasiswa lulus (`NA >= 60`) menggunakan `if`.

 Kumpulkan kode program Anda beserta screenshot output ke sistem pengumpulan tugas sebelum pertemuan berikutnya.

Terima Kasih!

Semoga materi Pertemuan 2 ini bermanfaat dan mudah dipahami. Teruslah berlatih menulis kode Java setiap hari — semakin sering berlatih, semakin mahir Anda!



Yang Sudah Dipelajari

- Struktur dasar program Java
- Output dengan `println` & `printf`
- Input menggunakan `Scanner`
- Variabel dan konsep memori
- Operator aritmatika & relasional
- Pernyataan `if` untuk keputusan

Pertemuan Berikutnya

- Struktur kontrol: `if-else`, `switch`
- Perulangan: `while`, `for`, `do-while`
- Metode dan modularisasi program

"Code is like humor. When you have to explain it, it's bad." — Cory House. Jadi, biasakan menulis kode yang bersih, terbaca, dan terdokumentasi dengan baik! 💡